

MOBILE HARNESSES FOR AI CODING ON PHONES

Anonymous authors

Paper under double-blind review

ABSTRACT

Large language model agents increasingly rely on execution harnesses: controlled environments that expose tools, preserve traces, validate artifacts, and report failures. On desktop and server platforms, such harnesses are often implemented as IDE integrations, browser environments, or remote sandboxes. Phones, however, remain an awkward substrate for AI coding: they have constrained screens, fragmented file entry points, restrictive execution environments, and mobile-specific sharing and preview flows. This paper argues that phone-native AI coding should be framed not as “putting a full IDE on a phone”, but as building a mobile harness: a control plane that coordinates model calls, local files, previews, runtime selection, repository delivery, and verifiable evidence under mobile constraints. We present MOBILEHARNESS, a phone-native AI coding harness prototype, and MOBILEHARNESSBENCH, a benchmark protocol for evaluating whether a mobile coding agent can close the loop from incoming files and code edits to preview, delivery, runtime orchestration, and evidence reporting. MOBILEHARNESSBENCH contains 25 seed tasks, a 1,000-task candidate bank across six mobile coding categories, mobile-specific test tiers, and a verifier contract. We validate the benchmark machinery with deterministic T0 dry runs: a five-task representative run (4 passed, 1 typed GitHub block) and a 60-task `smoke-v2` run (50 fixture-level passes, 10 typed GitHub blocks). The draft is therefore a scoped system-and-benchmark contribution, not a completed Android/iOS performance study: it contributes a harness abstraction, a task/verifier protocol, initial T0 evidence, and a device-tier plan for Android/iOS devices, simulators, and GitHub sandbox delivery.

1 INTRODUCTION

AI coding agents are often discussed as if progress comes from model intelligence alone. In practice, the unit that turns a model into a useful agent is the *harness*: the environment that exposes tools, preserves state, constrains actions, records traces, validates artifacts, and decides whether a task is complete. On-device AI apps are moving from chat demos toward task, skill, tool, and benchmark surfaces; we specialize that shift for mobile coding. Software-engineering benchmarks such as SWE-bench require repository state and executable tests (Jimenez et al., 2024); web and operating-system benchmarks such as Mind2Web, WebArena, and OSWorld require interactive environments and grounded observations (Deng et al., 2023; Zhou et al., 2024; Xie et al., 2024); mobile benchmarks such as Android in the Wild, AndroidWorld, MobileAgentBench, and PhoneWorld require phone-like interfaces, tasks, and verification signals (Rawles et al., 2023; 2025; Wang et al., 2024; Tang et al., 2026).

The phone is the missing substrate in this progression for AI coding. People increasingly encounter coding-adjacent work on phones: a malformed file arrives through a chat app, a Markdown report needs previewing, a generated HTML artifact needs inspection, a small patch needs review, or a cloud build needs to be triggered while away from a desktop. These tasks are not served well by compressing a desktop IDE into a small screen. Their failures are mobile failures: share sheets, document pickers, sandboxed files, WebViews, app backgrounding, permission prompts, unstable connectivity, and constrained local execution.

This motivates a sharper formulation: mobile AI coding should be studied not as “putting a full IDE on a phone”, not as general phone-use control, and not as model-only text generation, but as a phone-resident *control plane*. That plane receives messy inputs, invokes models, manages artifacts, selects

Line of work	Primary question	Missing for mobile coding
SE agents	Can an agent modify a repository and satisfy tests?	Usually assumes desktop/server execution, broad filesystem access, and non-mobile artifact handling.
Web/OS agents	Can an agent operate a browser or desktop environment?	Does not model mobile share flows, document pickers, WebViews, app lifecycle, or phone permissions.
Phone-use agents	Can an agent operate mobile apps through GUI actions?	Focuses on app manipulation rather than coding artifacts, repository delivery, runtime routing, or verifier reports.
Coding worlds	Can an agent write code against controlled APIs and app state?	Provides strong programmatic evaluation, but not phone OS boundaries or phone-native preview/delivery surfaces.

Table 1: Mobile coding harnesses sit between coding-agent benchmarks, phone-use environments, and practical mobile development workflows.

execution routes, previews outputs, delivers changes to remote infrastructure, and exports evidence. We call this layer a *mobile coding harness*: it separates model intelligence from mobile execution and evidence control while making phone-specific phenomena first-class evaluation targets rather than incidental UI details.

We study this problem through MOBILEHARNESS, a prototype phone-native AI coding harness. The prototype keeps model providers remote when needed, delegates heavyweight builds to external systems such as continuous integration, and keeps the user-facing control loop on the phone. The phone-side app coordinates chat state, tool traces, local project files, HTML/Markdown previews, runtime-provider selection, GitHub-oriented delivery, and report cards.

This paper makes three contributions. First, it defines the *mobile coding harness*: a phone-native control plane that makes model calls, artifacts, previews, delivery routes, and verifier evidence auditable under mobile OS constraints. Second, it introduces MOBILEHARNESSBENCH, a benchmark protocol and candidate-bank construction method for six phone-native coding categories: file intake, code editing, preview verification, GitHub delivery, harness evidence, and runtime orchestration. Third, it specifies an evidence-gated evaluation protocol covering T0 fixture validation, Android emulator and real-device tiers, Mac iOS simulator and iOS real-device tiers, GitHub sandbox delivery, and baseline comparison.

Scope of evidence. We distinguish task supply from task completion. The 1,000-task bank is a candidate bank, not a claim of 1,000 completed experiments. A task is counted only when verifier results, traces, artifacts, summaries, and required mobile-tier evidence are present; current evidence is T0-only over five representative tasks and a 60-task `smoke-v2` subset.

2 MOTIVATION: THE MISSING MOBILE HARNESS LAYER

Existing agent benchmarks illuminate different parts of the landscape, but none directly captures the phone-native AI coding loop. Table 1 summarizes the gap.

Gap criterion. In shorthand, $MCH = \text{PhoneOS} \cap \text{CodingArtifact} \cap \text{RouteControl} \cap \text{VerifierEvidence}$: software-engineering benchmarks usually relax PhoneOS, phone-use benchmarks usually relax coding-artifact and route-control requirements, and development tools are not evidence-gated benchmark protocols.

Desktop assumptions break on phones. Desktop coding assumes broad filesystem access, stable windows, keyboard-heavy interaction, long-running processes, and direct terminals. Phones instead mediate entry through share sheets, document pickers, app sandboxes, permissions, lifecycle events, and network changes; execution may be absent, delegated, or restricted to preview-only routes.

Coding on phones is artifact-centered. The most plausible near-term mobile coding tasks are small but consequential artifact workflows: opening an unknown file from a messaging app, repairing malformed JSON, generating static HTML, previewing Markdown, checking a WebView, preparing a public-safe report, or triggering a remote build. They matter because they occur at the edge of ordinary mobile use, where switching to a desktop interrupts the work.

The harness is the research object. For a mobile coding agent, the useful artifact is often not a compiled binary. It may be a generated HTML page, a Markdown report, a small configuration repair, a GitHub commit, a Pages URL, an Actions artifact, or a trace explaining why a task was blocked. The harness must therefore expose:

1. file intake and type recovery from mobile entry points;
2. local artifact creation and scoped editing;
3. WebView/Markdown preview and snapshot summaries;
4. runtime-provider selection and fallback;
5. repository delivery through authenticated APIs; and
6. verifier results and public-safe reports.

A mobile benchmark must count evidence, not intention. An offline fixture runner is useful for validating schemas and verifiers, but it cannot prove share-sheet behavior, real WebView rendering, background recovery, low-memory resilience, or iOS Open In behavior. Accordingly, MOBILEHARNESSBENCH separates candidate task definitions from counted benchmark results. Tasks become paper-counted only after producing verifier outputs, traces, summaries, and evidence from the required mobile test tier.

Definition. A *mobile coding harness* is the phone-resident control layer that binds four elements into one reproducible loop: mobile input surfaces, coding artifacts, execution or delivery routes, and verifier evidence. This definition intentionally excludes model weights and heavyweight build infrastructure. Those components may be remote; the harness is responsible for making their use observable, recoverable, and reviewable from the phone. We formalize the harness as

$$\mathcal{H} = (\mathcal{I}, \mathcal{C}, \mathcal{P}, \mathcal{R}, \mathcal{D}, \mathcal{E}, \mathcal{V}),$$

where $\mathcal{I}$ is mobile input intake, $\mathcal{C}$ is artifact construction and editing, $\mathcal{P}$ is preview, $\mathcal{R}$ is runtime routing, $\mathcal{D}$ is delivery, $\mathcal{E}$ is evidence capture, and $\mathcal{V}$ is verification. A run is a state transition $x_{k+1} = \mathcal{H}(x_k, u_k, o_k)$, where u_k is the user or model action and o_k is the mobile observation. The run is reviewable only when it emits an evidence certificate

$$\zeta_t = (a_t, p_t, \tau_t, m_t, v_t),$$

where a_t is the produced artifact set, p_t is preview state, τ_t is the trace, m_t is mobile-tier evidence, and v_t is the verifier result.

3 SYSTEM: A PHONE-NATIVE CODING HARNESS

Our prototype is a phone-native application organized around a harness control plane rather than a local IDE clone. Figure 1 summarizes the architecture.

Formal loop. Operationally, the harness composes six modules:

$$\mathcal{H} = \mathcal{V} \circ \mathcal{L} \circ \Phi \circ \Omega \circ \Pi \circ N_{\mathcal{I}},$$

where $N_{\mathcal{I}}$ normalizes mobile inputs, Π plans artifact actions, Ω selects a runtime or delivery route, Φ renders previews, $\mathcal{L}$ records evidence, and $\mathcal{V}$ verifies the run. The typed interfaces are $N_{\mathcal{I}} : i \mapsto \hat{i}$, $\Pi : (\hat{i}, g) \mapsto (a, \nu)$, $\Omega : (t, \mathcal{R}_t) \mapsto r^*$, $\Phi : a \mapsto p$, $\mathcal{L} : (a, p, r^*) \mapsto \tau$, and $\mathcal{V} : (t, a, p, \tau, m) \mapsto (s, c)$. The route policy balances verifiability, evidence capture, authorization, locality, cost, and risk:

$$r_t^* = \arg \max_{r \in \mathcal{R}_t} \lambda_v q_v(r, t) + \lambda_e q_e(r, t) + \lambda_a q_a(r, t) + \lambda_l q_l(r, t) - \lambda_c c(r, t) - \lambda_\rho \rho(r, t).$$

The coefficients are policy parameters, not learned claims in this draft; their purpose is to make route choices auditable.

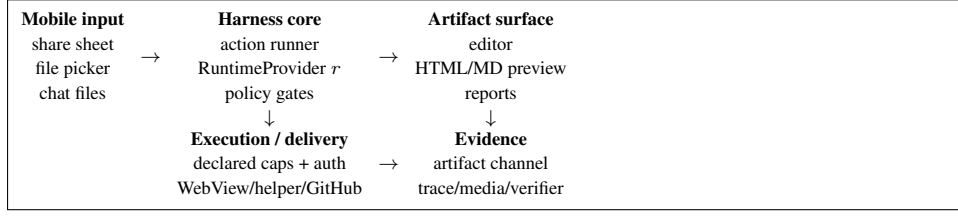


Figure 1: Module view of a mobile coding harness. The RuntimeProvider boundary makes route capability, authorization, artifact flow, and evidence flow explicit while the phone keeps the control loop.

Design invariants. The control plane enforces route explicitness, evidence monotonicity, and public-report safety:

$$\mathcal{I}_{\mathcal{H}}(t) = \mathbf{1}[\text{route}(t) \wedge (\tau_{k+1} = \tau_k \oplus e_k) \wedge \text{safe}(\text{report}_t)].$$

Every action records its selected route and fallback reason, evidence is appended rather than overwritten, and exported reports must be redacted or repo-relative.

Algorithm 1: Mobile harness execution loop. **Input:** mobile request t , current state x , provider set $\mathcal{R}_t$. **Output:** evidence certificate ζ_t , status s .

1. Normalize incoming file, share target, or chat artifact through $N_{\mathcal{I}}$.
2. Plan artifacts and verifier ids with Π ; choose r_t^* with Ω .
3. Execute edit, preview, or delivery; emit typed blocks when prerequisites are absent.
4. Record trace/media/report evidence with $\mathcal{L}$, then verify with $\mathcal{V}$.
5. Return $\zeta_t = (a_t, p_t, \tau_t, m_t, v_t)$ and status s .

RuntimeProvider boundary. RuntimeProvider is the executable module contract $r = (\text{cap}, \text{auth}, \text{exec}, \text{artifact}, \text{evidence}, \text{block})$, admitted only when $\text{admit}(r, t) = \mathbf{1}[\kappa_t \subseteq \text{cap}(r) \wedge \text{auth}(r, t) \wedge \text{safe}(r, t)]$. ActionRunner executes only admitted routes; EvidenceLedger appends traces, media pointers, verifier outputs, and safe summaries. The prototype instantiates helper, external-terminal, WebView-only, and future cloud-backed routes; unavailable prerequisites produce typed `blocked` states rather than ad-hoc execution.

Preview-first artifacts. HTML and Markdown are first-class artifacts: they can be generated, read back, previewed in WebView, exported, and delivered through repository-backed static hosting. The harness records preview routes, snapshot metadata, and failure kinds instead of relying on visual inspection alone.

GitHub-first delivery. Mobile coding rarely benefits from local heavyweight builds, so repository APIs, Pages hosting, Actions dispatch, and artifact discovery are delivery surfaces. Authenticated actions are evaluated separately; unavailable authorization marks a GitHub-delivery task `blocked`, not failed.

Evidence and reporting. Each action should produce evidence: inputs, tool actions, outputs, artifacts, preview URLs, logs, failure kinds, and recovery suggestions. Public reports must use redacted or repo-relative evidence and must not expose private local paths or credentials.

4 MOBILEHARNESSBENCH: A BENCHMARK FOR MOBILE CODING HARNESSES

MOBILEHARNESSBENCH is a benchmark protocol for testing whether a phone-native harness can close the loop from user intent to verified artifact under mobile constraints. It is not a general phone GUI benchmark and does not ask whether an agent can operate arbitrary consumer apps; it evaluates a phone-native coding harness. It focuses on AI coding workflows that naturally occur on phones.

Category	Count in v2	What it tests
File intake	167	Share/file-picker entry, type detection, fallback preview, unsupported files
Code edit	167	Write/read/patch workflows, JSON repair, Markdown report generation, scoped diffs
Preview verification	167	HTML/Markdown preview, blank/invalid preview detection, local image handling
GitHub delivery	167	Commit metadata, conflict handling, Pages publish, Actions dispatch, artifacts
Harness evidence	166	Complete traces, typed failures, redaction, recovery, run comparison
Runtime orchestration	166	Runtime health, fallback, WebView-only mode, provider switching, task stop

Table 2: MOBILEHARNESSBENCH v2 candidate task bank contains 1,000 candidate tasks across six mobile coding harness categories.

4.1 TASK CATEGORIES

Table 2 lists the six categories.

4.2 TASK DEFINITION

Each task is a machine-readable tuple

$$t = (id, c, g, i, \kappa, a, \nu, e, b, \mu, o),$$

where c is category, g is the user goal, i is the input fixture, κ are required capabilities, a are expected artifacts, ν are verifier ids, e are evidence requirements, b are blocked conditions, μ is the mobile profile, and o is the oracle. A run returns

$$y_t = (s, \zeta_t, \delta_t),$$

where $s \in \{\text{passed}, \text{warning}, \text{failed}, \text{blocked}\}$, ζ_t is the evidence certificate, and δ_t is the typed explanation for non-counted or non-successful results. The tuple makes construction auditable: goals and fixtures define supply, verifier and oracle fields define evidence, and mobile profiles plus tier targets define counting gates:

$$\mathcal{B} = \text{Assemble}(C, F, M, Q, O, V, T)$$

with categories C , fixture families F , mobile profiles M , quality axes Q , oracles O , verifier contracts V , and target evidence tiers T .

Algorithm 3: Differentiated candidate-bank construction. The generator is a constrained sampler: (1) stratify target counts over C, F, M, Q, O ; (2) generate $t_j = G(c_j, f_j, m_j, q_j, o_j)$ with required capabilities, artifacts, verifier ids, blocked conditions, and safety markers; (3) reject same-category near-duplicates when $\Delta(t_i, t_j) = d_g(g_i, g_j) + d_f(f_i, f_j) + d_m(m_i, m_j) + d_o(o_i, o_j) < \epsilon$; (4) audit balance, unique titles/goals, oracle completeness, mobile-profile coverage, and tier declarations; and (5) freeze only manifest-selected subsets, leaving performance counts to evidence-gated runs. The v2 audit checks 1,000 unique titles, 1,000 unique user goals, category balance, quality-axis and mobile-profile coverage, task-set manifests, required oracle fields, and public-output safety markers, while preserving human review and mobile-device execution as open requirements.

4.3 QUALITY AXES

The v2 candidate bank assigns each task a coverage vector

$$q(t) = (q_{\text{happy}}, q_{\text{recovery}}, q_{\text{safety}}, q_{\text{mobile}}) \in \{0, 1\}^4.$$

The four coordinates mark happy-path completion, typed failure recovery, public-report safety, and declared mobile constraints such as viewport, app state, and resource limits. They are stratification tags for construction and audit, not difficulty estimates or performance claims.

Tier	Environment	Role
T0	Offline fixture runner	Schema, fixture, verifier, report safety
T1	Android emulator	UI/WebView regression and log capture
T2	Android real device	Share/Open-with, WebView, lifecycle, low-memory evidence
T3	Mac iOS simulator	iOS WebView, document picker, Xcode-log regression
T4	iOS real device	Open In, Files app, permissions, lifecycle evidence
T5	GitHub sandbox	Commit, Pages, Actions, artifact delivery

Table 3: MobileHarnessBench separates candidate tasks from counted results by requiring evidence from the relevant mobile tier.

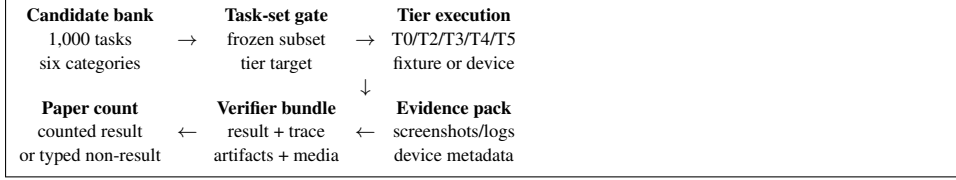


Figure 2: Evidence promotion flow. Scale creates coverage; only verifier-complete tier evidence creates counted benchmark results.

4.4 MOBILE TEST TIERS

Table 3 gives the evaluation tiers. The key design principle is that offline tests are necessary but insufficient. Each task declares required evidence tiers $R_t \subseteq \{T0, \dots, T5\}$. For run r , tier sufficiency is $\text{tier}(t, r) = \mathbf{1}[R_t \subseteq E_r \wedge M_r \wedge L_r \wedge A_r \wedge D_r]$, where E_r is observed tier, M_r media, L_r logs/traces, A_r artifacts, and D_r device or sandbox metadata. This predicate prevents T0 or simulator-only evidence from being counted when a task requires real-device behavior.

Figure 2 shows the evidence-promotion rule used by the benchmark artifacts. A candidate task can be schema-valid and still remain non-result evidence until the required tier artifacts are attached.

Algorithm 4: Mobile-tier promotion audit. Normalize E_r, M_r, L_r, A_r, D_r ; reject missing required fields with typed reasons; downgrade emulator-only or simulator-only runs when R_t marks real-device behavior; run verifier contracts; apply public-report redaction; count only when $\pi(t, r) = 1$.

Simulator/real-device boundary. Android emulators and Mac iOS simulators provide regression evidence for layout, WebView preview, document picker behavior, Xcode logs, and repeatable UI traces, but they cannot stand in for T2/T4 tasks that depend on file sharing, Open with/Open In flows, lifecycle, permissions, memory pressure, external apps, or network behavior.

5 VERIFIER CONTRACT

MOBILEHARNESSBENCH uses verifiers to turn “looks done” into structured evidence. A machine-readable verifier catalog defines each verifier id, required inputs, required evidence, pass conditions, failure kinds, category scope, and current T0 support status. A verifier result contains:

- task id, status, score, and failure kind;
- named checks with passed, warning, failed, or blocked status;
- artifact paths, preview URLs, screenshots, and logs;
- recovery suggestions for blocked or failed cases.

Statuses have explicit semantics. `passed` means the user goal and required verifiers succeeded. `warning` means the core goal completed with non-blocking issues. `failed` means a required artifact, verifier, or safety condition failed. `blocked` means an external permission, account, platform, runtime, or device capability prevented execution and should not be counted as model or harness failure. Failure kinds are grouped into artifact/preview failures, trace/evidence

Run	Tasks	Passed	Blocked	Failed	Evidence boundary
representative-v0	5	4	1	0	pipeline check across the original five categories
smoke-v2	60	50	10	0	fixture-only T0 evidence; not Android/iOS device evidence

Table 4: Deterministic offline dry runs. These validate task fixtures, reports, traces, typed blocking, and safety checks, but they are not full mobile-device experiments.

failures, runtime-provider failures, authorization or delivery blocks, privacy/public-output failures, and missing mobile-tier evidence. For example, `blank_preview`, `trace_missing`, `runtime_provider_missing`, `github_auth_blocked`, and `public_path_leak` identify different recovery paths and should not be collapsed into one generic error.

Evidence-gated verifier module. The verifier can be viewed as a small decision module that maps a task run into a status and a counted-evidence flag. Let V_t denote required verifier checks, R_t trace completeness, A_t artifact availability, Q_t public-output safety, D_t whether the task requires real-device evidence, and M_t whether the required mobile-tier evidence is present. A task is counted in the paper only if

$$\text{counted}(t) = \mathbf{1}[V_t \wedge R_t \wedge A_t \wedge Q_t \wedge (\neg D_t \vee M_t)].$$

This rule is intentionally stricter than task generation: candidate tasks increase coverage, while counted tasks require evidence.

Algorithm 2: Evidence-gated verifier module. **Input:** task t , trace r , artifacts a , tier evidence m .

Output: status s , counted flag c .

1. Validate required artifacts, preview state, trace coverage, and public-output safety.
2. If an external permission, account, runtime, or device prerequisite is missing, set $s = \text{blocked}$.
3. Else if any required verifier check fails, set $s = \text{failed}$; otherwise set $s \in \{\text{passed}, \text{warning}\}$.
4. Set $c = \text{counted}(t)$ using the evidence gate above and preserve the typed reason when $c = 0$.

The current offline verifier implementation covers the five-task representative run and the 60-task smoke-v2 T0 run. It checks external file intake, generated HTML artifacts, preview summaries, deterministic JSON repair, GitHub delivery blocking, trace completeness, runtime metadata, and public-output safety. GitHub-delivery tasks are intentionally marked `blocked` in T0 runs because no remote write is performed.

6 PRELIMINARY EVIDENCE

The artifact package uses an evidence-state map $E(a) \in \{\text{SUPPLY}, \text{T0}, \text{OPEN}\}$. `SUPPLY` covers the 25-task seed set and 1,000-task v2 bank; `T0` covers deterministic fixture checks, quality audits, and offline dry runs; `OPEN` covers real Android/iOS runs, GitHub sandbox delivery, and counted baseline comparisons. This map is part of the benchmark protocol: scale is reported as task supply, while performance claims require verifier-complete evidence. The claim-evidence ledger is a readiness module $\Gamma_j = (\text{claim}_j, A_j, \eta_j)$ with $\text{ready}(\Gamma_j) = \mathbf{1}[\forall a \in A_j : E(a) \succeq \eta_j \wedge \text{safe}(a)]$; the evidence maturity matrix prevents T0 fixtures, readiness probes, and baseline scaffolds from being promoted to mobile or baseline results.

Table 4 summarizes the current T0 offline evidence.

Each run produces a JSON report, a Markdown summary, JSONL traces, and task-specific artifacts. The public run output is constrained to repo-relative paths and synthetic preview routes. The validator checks that task banks have the expected category distribution, that task-set manifests refer to valid tasks, that run artifacts exist, that traces match result task ids, and that public outputs avoid sensitive markers.

7 EVALUATION PROTOCOL

The evaluation is artifact-bound across task-bank construction, T0 verifier validation, mobile-device execution, delivery validation, and baselines. The unit is a trial record $\rho_t = (t, w, r, e_t, s_t, c_t)$, where w is workflow, r route, s_t status, c_t counted flag, and $e_t = (\zeta_t, \ell_t, d_t, h_t)$ stores certificate, logs/media, device or sandbox metadata, and human-intervention evidence. The promotion gate $\pi(t, r) = \mathbf{1}[\text{schema}(t) \wedge \text{verifier}(r) \wedge \text{tier}(t, r) \wedge \text{safe}(r)]$ admits counted harness results only when schema, verifier, tier, and safety gates pass; for baselines the counted flag also requires B_w . All other outputs remain task supply or typed non-results.

E1: T0 smoke over v2. Run the 60-task `smoke-v2` subset, ten tasks per category, using offline verifiers where possible. Report task success, verified success, blocked rate, trace completeness, and report-safety pass rate.

E2: Android real-device subset. Run the 30-task `android-device-v2` subset on at least one Android real device. Record device metadata, screenshots or recordings, logcat excerpts, run JSON, summaries, and traces for file sharing, Open with, WebView, runtime orchestration, and lifecycle evidence.

E3: Mac iOS simulator subset. Run the 18-task `ios-simulator-v2` subset on a Mac iOS simulator. Record simulator screenshots, Xcode logs, viewport metadata, traces, and summaries; this is a regression layer and does not replace a real iPhone study.

E4: GitHub sandbox delivery. Run authorized GitHub-delivery tasks against a public sandbox repository. Record commit SHAs, Pages URLs, Actions run URLs, artifact metadata, and typed blocked cases.

E5: Baseline comparison. Compare three workflows: chat-only mobile coding, desktop remote IDE flow, and the phone-native harness flow. All baselines use the same frozen task subset, input fixtures, authorization state, time budget, and human-intervention logging. For workflow w , a baseline result can be counted only if

$$B_w = L_w \wedge P_w \wedge A_w \wedge V_w \wedge H_w,$$

where L_w is a model/provider lock, P_w prompt transcript completeness, A_w artifact or typed-block evidence, V_w verifier output, and H_w human-intervention annotation. The artifact package defines this protocol, schema, pilot prompts, and readiness gate, but `counts_as_experiment=false`; scaffold, dry-run, capture-ready, and pilot-ready files remain preparation, not performance results.

Primary metrics. For a task set T , we report rate metrics with denominator $|T|$: $\text{TaskSuccess} = |\{t : g_t = 1\}|/|T|$, $\text{VerifiedSuccess} = |\{t : g_t = 1 \wedge V_t = 1\}|/|T|$, $\text{TraceCompleteness} = |\{t : R_t = 1\}|/|T|$, $\text{RecoveryRate} = |\{t : b_t = 1 \wedge r_t = 1\}|/|\{t : b_t = 1\}|$, and $\text{ArtifactAvailability} = |\{t : A_t = 1\}|/|T|$, where g_t is goal completion, V_t verifier success, R_t trace completeness, b_t blocked or failed status, r_t actionable recovery guidance, and A_t reviewable artifacts. For baseline w , the reported vector is

$$\mathbf{m}_w = (\text{TaskSuccess}, \text{VerifiedSuccess}, \text{TraceCompleteness}, \text{RecoveryRate}, \text{ArtifactAvailability}, \bar{h}, \bar{n}),$$

where $\bar{h}$ and $\bar{n}$ are mean human interventions and mean steps to completion.

8 RELATED WORK

Adjacent evaluation objects. SWE-bench and AppWorld make repository or world-state repair verifier-first (Jimenez et al., 2024; Trivedi et al., 2024); Mind2Web, WebArena, and OSWorld evaluate grounded operation in web or desktop-like environments (Deng et al., 2023; Zhou et al., 2024; Xie et al., 2024); Android in the Wild, AndroidWorld, MobileAgentBench, and PhoneWorld move grounded operation onto phones (Rawles et al., 2023; 2025; Wang et al., 2024; Tang et al., 2026).

Their objects are repository repair, environment control, and phone-app control; MOBILEHARNESSBENCH instead measures a phone-resident coding harness: mobile intake, coding artifacts, route choice, and verifier evidence under explicit device-evidence boundaries.

Tools as evidence surfaces. Android Studio, Xcode, Flutter tooling, emulators, simulators, device bridges, CI systems, GitHub Actions, and static hosting are useful infrastructure, but not benchmark protocols. They provide admissible signals such as logs, screenshots, commits, URLs, artifacts, and build metadata. MOBILEHARNESSBENCH treats these tools as evidence surfaces rather than baselines to imitate: a result counts only when a phone-resident certificate links intake, artifacts, route choice, preview or delivery, verifier output, and public-safe reporting.

9 CONCLUSION

MOBILEHARNESSBENCH frames phone-native AI coding as a mobile harness problem: the phone hosts control, preview, delivery, and evidence rather than imitating a desktop IDE; it then turns that frame into an evidence-gated benchmark protocol with differentiated task supply, tiered mobile evidence, typed non-results, and reviewer-visible claim boundaries. The remaining work is empirical: attach real Android/iOS evidence, authorized GitHub sandbox delivery, and counted baselines before promoting beyond T0 evidence.

10 LIMITATIONS AND THREATS TO VALIDITY

The main construct/internal threat is that the 1,000-task v2 candidate bank is not a completed benchmark release or a general phone-use benchmark; T0/offline runs validate fixtures, traces, verifier coverage, route logging, and report safety, not Android/iOS share sheets, WebViews, lifecycle, permissions, or runtime behavior. External, baseline, privacy, and submission threats remain open gates: device/OS diversity, GitHub sandbox authorization, locked baseline runs, public-output leakage checks, and venue/template/OpenReview/bibliography drift.

11 REPRODUCIBILITY STATEMENT

The package is reproducible as a draft/T0 artifact: it includes machine-readable task definitions, fixtures/manifests, a machine-readable verifier catalog, a deterministic task-bank generator, an offline verifier runner, a validator, and commands for the claim-evidence ledger, evidence maturity matrix, evaluation-protocol, method-presentation, verifier-contract, submission-readiness, page-limit, and reproducibility readiness reports. It is not yet reproducible as a full empirical result: the anonymous supplement is a boundary artifact, not extra results; mobile-tier readiness is non-experimental because Android adb and Mac/Xcode are absent, and the frozen subset remains `counts_as_final_paper_subset=false` until mobile and GitHub-sandbox evidence are attached. Future experimental runs will add device metadata before promotion.

12 ETHICS STATEMENT

The main ethical risks are privacy leakage, credential exposure, and misleading benchmark claims; MOBILEHARNESSBENCH handles them through public-report safety, typed authorization blocking, private-repo exclusion, mobile-evidence counting gates, no human-subject data, and human responsibility for claims.

REFERENCES

- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.06070>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth*

- International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the Wild: A large-scale dataset for android device control. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2307.10088>.
- Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William E. Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy P. Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=il5yUQsrjC>.
- Zhengyang Tang, Yuxuan Liu, Xin Lai, Junyi Li, Pengyuan Lyu, Jason, Yiduo Guo, Zhengyao Fang, Yang Ding, Yi Zhang, Weinong Wang, Huawei Shen, Xingran Zhou, Liang Wu, Fei Tang, Sunqi Fan, Shangpin Peng, Zheng Ruan, Anran Zhang, Benyou Wang, Rui Yan, Ji-Rong Wen, Chengquan Zhang, and Han Hu. PhoneWorld: Scaling phone-use agent environments. *arXiv preprint arXiv:2605.29486*, 2026. URL <https://arxiv.org/abs/2605.29486>.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.
- Luyuan Wang, Yongyu Deng, Yiwei Zha, Guodong Mao, Qinmin Wang, Tianchen Min, Wei Chen, and Shoufa Chen. MobileAgentBench: An efficient and user-friendly benchmark for mobile llm agents. *arXiv preprint arXiv:2406.08184*, 2024. URL <https://arxiv.org/abs/2406.08184>.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2404.07972>.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2307.13854>.